

Project

Port Database

Your company, **Apasaja Private Limited**, has been commissioned to design and implement a database application to manage container operations at the Persinggahan terminal. The application stores and manages information about the ships and containers currently in the terminal.

The application stores and manages information about containers. The containers are either in the terminal's yards or on ships docked at the terminal. A container is identified by its ISO standard (ISO 6346) identification [1]. The application also stores a text description of the containers' content. For instance, the container with the identification number MSCU5285725 is "empty".

The application also stores and manages information about the companies that own containers. Ownership is indicated by the first three letters of the container's identification code. The application records the company's name, address, postal code, city, and country. For example, the container MSCU5285725 is owned by "MSC - Mediterranean Shipping Company S.A.", located at Chemin Rieu 12-14, CH 1208, Geneva, Switzerland. The application maintains company information, including companies without containers currently at the terminal. For convenience, the application stores and manages information about companies and their cities and countries even if they do not have containers currently in the terminal.

As the database is to be used world-wide and to attract future clients, there may be cities recorded without any companies in them yet. There may also be countries recorded without any recorded cities within the countries. Given the world-wide nature of the database, there may be cities with the same name but located in different countries. For instance, there are 52 cities named San Antonio in 16 countries across 3 continents. These information should be stored in the database.

The application stores and manages details about the ships docked at the terminal. Each ship is identified by its Maritime Mobile Service Identity (MMSI) number [2], International Maritime Organisation (IMO) number [3], or call sign. The application also records the ship's name and dimensions. For instance, the ship with MMSI 563159100, IMO number 9339064, and call sign 9V8013 is PACIFIC NINGBO, a 129.57-metre-long and 20.83-metre-wide vessel.

The application stores and manages information about the berths in the terminal. Each berth is identified by code (a natural number) and has a position (the longitude and latitude of its centre [4]). For instance, the centre berth 42 is located at a longitude of 1.29662 degrees north and a latitude of 103.77643 degrees east. Five decimal places in the longitude and latitude identify the position with meter-level precision.

Each ship at the terminal occupies one berth, and each berth can hold only one ship at a time. Although there may be berth without a ship, the database should still record the information about the berth even if there is no ships docked in the berth. However, each ship must be docked at a berth. We need not record ships that are not docked in any berth.

The application tracks the locations of containers within the yards, which are categorized as normal, semi-automated, or automated. Each yard is identified by a unique code. Within each yard, container locations are specified by a bay number, row number, and tier number, all of which are natural numbers. Containers can be stacked, with tier number 1 indicating a position on the ground, and higher tier numbers indicating containers stacked on top of others in sequence. Each yard type have their own maximum number of tiers allowed. For instance, normal yards may have a maximum of five tiers, semi-automated yards may allow up to seven tiers, and automated yards may accommodate up to nine tiers. However, this data may be updated as the port may be upgraded.

Although the port is busy, there may still be some yards that has no container. Their information is to be recorded as well. Similarly, there may be some type of yards currently not fully (e.g., AI-enabled yard) so it currently has no yards yet but still recorded in the database. Furthermore, the yard mmay be newly created and has no position available for container storage yet. Again, the information must still be recorded.

The application stores and manages the current state of the terminal, does not keep historical records, and does not manage future events. Note that although some information about the port may not be available, they may still be recorded. For instance, we may still want to know which type of yard currently has no yard.

1. Entity-Relationship Modelling [5 Points]

- **Entity-Relationship Model** Draw the entity-relationship diagram for the application. Show all entity sets, relationship sets, attributes, participation constraints, etc. Clearly indicate the identities. Use the notation from the lecture.

  Explain how the entity-relationship diagram enforces the constraints in the specification. Justify your design with the statements from the specification. List constraints that are not enforced by the entity-relationship diagram (if any).

  Important Note

  - Write your group number in the report.
  - We do not accept hand-drawn diagrams.
    - You may use your favorite online diagram tools.
    - You may use the templates in `ERD_notations.pptx` to get started.
    - You may draw the diagram by hand first before converting it to a digital drawing before submission.
  - Use at least **12pt** font size for legibility.
    - Marks may be deducted for illegible diagrams (e.g., the font is deemed too small to be read).
    - The font size should be used for all text including names of attributes, entity sets, relationship sets, etc.
  - Ensure that your diagram is unambiguous.
    - Marks may be deducted for ambiguous diagrams (e.g., lines crossing, non-meaningful names, etc).
    - We reserve the right to always choose the incorrect meaning when the diagram is ambiguous.
  - Avoid surrogate/artificial keys.
  - Your entity-relationship relationship diagram should fit into one A4 page.
    - You still need to satisfy the font size requirement and unambiguity.
  - You may or may not be able to enforce all constraints.
    - You need to enforce as many constraints as possible.
    - For any constraints not enforced, list them on your submission.
  - You do not have to replicate the columns structure in `companies.csv`, `containers.csv`, or `ships.csv` exactly.
    - You are encouraged to transform the data (e.g., splitting columns, adding/removing columns, etc).
    - They are given to provide an example. The data may be incorrect and may violate some constraints from the specification.

  Submission Instruction

  - Submit your solution on Canvas.
    - `Assignments` ▶ `Project 1: Entity-Relationship Diagram`
  - Submit the following file.
    - `erd.pdf` containing your entity-relationship diagram and its explanation.

2. Schema and Query [10 Marks]

- **ERD** Update your entity-relationship diagram to address the comments.

  Important Note

  > - Any issues not addressed will be heavily penalized in this submission.

  Submission Instruction

  > - Submit your solution on Canvas.
  >   - `Assignments` ▶ `Project 2: Entity-Relationship Diagram`
  > - Submit the following file.
  >   - `erd.pdf` containing your entity-relationship diagram and its explanation.

- **Schema** Translate the entity-relationship diagram into schema using only the 5 basic constraints: `PRIMARY KEY`, `UNIQUE`, `NOT NULL`, `FOREIGN KEY`, and `CHECK`.

  Construct tables using SQL data definition language (i.e., `CREATE TABLE` statement). You may use `ALTER TABLE` if necessary. You should use deferrable constraints [5] if possible. However, you are not allowed to use other statements including but not limited to the following.

  - `CREATE DATABASE`: You should use the default database.
  - `CREATE SCHEMA`: You should use the default schema `public`. You should not alter `search_path`.
  - `CREATE TYPE`, `CREATE ENUM`, etc: Use the best fit standard type [6]. All data should be atomic.
  - `CREATE FUNCTION`, `CREATE PROCEDURE`, `CREATE TRIGGER`, etc: These will be used in the future.

  Justify any event actions (if needed, as comment on SQL). Further justify any deviations from the standard schema translation from the lecture. Write your justification as comments on the code.

  Important Note

  > - Do not add `DROP TABLE` statements.
  >   - Assume that we start from a blank database.
  > - Avoid surrogate/artificial keys.
  > - Use meaningful names from your entity-relationship diagram.
  > - Justify event actions.
  > - You may or may not be able to enforce all constraints.
  >   - You need to enforce as many constraints as given by the description as possible.
  >   - For any constraints not enforced, list them on your submission (as comment on SQL).
  > - You do not have to replicate the csv columns structure exactly.
  >   - You are encouraged to transform the data (e.g., splitting columns, adding/removing columns, etc).

  Submission Instruction

  > - Submit your solution on Canvas.
  >   - `Assignments` ▶ `Project 2: Schema`
  > - Submit the following file.
  >   - `schema.sql` containing your data definition language.

- **(Query)** Perform the following set of operations as SQL statements `INSERT INTO` and `DELETE FROM`. You may need to add all the other necessary information (e.g., city, country, etc) according to your schema. The operations must be performed in the order they are given below. Each operation may correspond to more than one SQL statements.

1. Create a single yard with code `"Y"` with a yard type `"Simple"`.
   - A yard type `"Simple"` should have a max tier of 3.
   - There should be 2 bays and 2 rows for this yard. You need to add these.
2. Create 2 berths with code `"B1"` and `"B2"`.
3. Add a company named `"Apasaja Distribution International"` with code `"ADI"`.
4. Ship (i) MMSI = `211382280`, (ii) IMO = `9229843`, and (iii) Call Sign = `"5LJY5"` arrives at berth `"B1"`.
   - It has a container with ISO = `"ADIU4974982"` belonging to the company with the code `"ADI"`.
5. Ship (i) MMSI = `209912000`, (ii) IMO = `9261889`, and (iii) Call Sign = `"5BLP5"` arrives at berth `"B2"`.
   - It has a container with ISO = `"ADIU7385836"` belonging to the company with the code `"ADI"`.
6. Move the container with the ISO = `"ADIU4974982"` to yard `"Y"` at bay number `0` and row number `0`.
7. Ship with MMSI = `211382280` leaves the berth.
8. Ship (i) MMSI = `255806008`, (ii) IMO = `9277400`, and (iii) Call Sign = `"3FIV6"` arrives at berth `"B1"`.
   - It has a container with ISO = `"ADIU7583471"` belonging to the company with the code `"ADI"`.
9. Ship with MMSI = `209912000` leaves the berth.
10. Move the container with the ISO = `"ADIU7583471"` to yard `"Y"` at bay number `0` and row number `0`.

Write an SQL query to find all unused positions in the yard. Output the yard code, yard type, bay number, row number, and tier number. Sort the output in ascending order of yard code, followed by ascending order of bay number, followed by ascending order of row number, and followed by ascending order of tier number.

The expected output is the following table.

yard_code	yard_type	bay_number	row_number	tier_number
Y	Simple	0	0	3
Y	Simple	0	1	1
Y	Simple	0	1	2
Y	Simple	0	1	3
Y	Simple	1	0	1
Y	Simple	1	0	2
Y	Simple	1	0	3
Y	Simple	1	1	1
Y	Simple	1	1	2
Y	Simple	1	1	3

Important Note

- You are not allowed to use other statements besides `INSERT INTO` or `DELETE FROM`.
- Each operation may correspond to more than one SQL statements (e.g., a `DELETE FROM` followed by an `INSERT INTO`).

Submission Instruction

- Submit your solution on Canvas.
  - `Assignments` ▶ `Project 2: Queries`
- Submit the following file.
  - **`query.sql`** containing your operations and your SQL query.

3. Stored Procedures and Triggers [10 Marks]

During the design of your Port Database using an Entity-Relationship Diagram (ERD) and then implementing the database by creating its schema, your goal was to capture as many application constraints as possible. For example, knowing that each ship is identified by its MMSI number, IMO number, or call sign, one attribute needed to be selected as the primary key of the table `ships` and the other attributes needed to be set to `UNIQUE NOT NULL`. However, you have noticed that some application constraints could not be captured by the schema using the 5 basic constraint types: `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `UNIQUE`, and `CHECK`. For this question, your task is therefore to implement a series of triggers to capture additional applications constraints.

- **(Schema)** Update your schema to satisfy the following.
    1. Address the feedback given.
    2. All constraints are deferrable where possible.
    3. All foreign keys must use `ON UPDATE CASCADE` and `ON DELETE CASCADE`.
    4. All primary keys are minimal, and all foreign keys refer to only primary keys.

    Important Note

    - Any feedback issues not addressed will be heavily penalized in this submission.
    - Schemas that do not follow the above requirements will also be heavily penalized.

- **(Triggers)** Add triggers to your updated schema to capture and preserve the following four constraints.
    1. There is no container that is neither on a ship nor in a yard.
    2. A container must either be on a ship or in a yard (but not both).
    3. A container in a yard cannot be stacked higher than the maximum tier of that yard.
    4. A container in a yard is either on the ground or on top of another container; there are no “floating” containers.

    Important Note

    - We recommend creating 4 triggers, one trigger for each constraint.
    - Before writing the triggers, think about how you can write an SQL query that tells if a constraint might be violated.
    - All triggers can be solved without the use of `OLD` or `NEW` (but they do not have to be).
    - Avoid unnecessary triggers. For example, a trigger `ON INSERT` even though insertions will never cause a violation, or a trigger on table `ships` even though this table does not affect the constraint being checked.
    - You should come up with your own test cases to check if your triggers indeed prevent you from modifying the database through `INSERT`s, `UPDATE`s, or `DELETE`s that might result in a state violating any of the four application constraints.
    - If an operation will lead to an inconsistent state, you should not modify the operation’s data to make it successful. Instead, you should throw exception [7].
    - Your triggers should always be deferrable. Constraints should only be checked at the end of the transaction [8].

    Submission Instruction

    - Submit your solution on Canvas.
        - `Assignments` ▶ `Project 3: Stored Procedures and Triggers`
    - Submit the following file.
        - **`triggers.sql`** containing your schema and triggers.

Bibliography

[1] “Wikipedia: ISO 6346.” [Online]. Available: https://en.wikipedia.org/wiki/ISO_6346

[2] “IMO identification number schemes.” [Online]. Available: https://www.imo.org/en/ourwork/msas/pages/imo-identification-number-scheme.aspx

[3] “Maritime Mobile Service Identities - MMSI.” [Online]. Available: https://www.fcc.gov/wireless/bureau-divisions/mobility-division/maritime-mobile/ship-radio-stations/maritime-mobile

[4] “Wikipedia: ISO 6709.” [Online]. Available: https://en.wikipedia.org/wiki/ISO_6709

[5] “PostgreSQL Docs: CREATE TABLE (Deferrable).” [Online]. Available: https://www.postgresql.org/docs/current/sql-createtable.html#SQL-CREATETABLE-PARMS-DEFERRABLE

[6] “PostgreSQL Docs: Date/Time Functions and Operators.” [Online]. Available: https://www.postgresql.org/docs/current/datatype.html

[7] “PostgreSQL Docs: Errors and Messages.” [Online]. Available: https://www.postgresql.org/docs/current/plpgsql-errors-and-messages.html

[8] “PostgreSQL Docs: Errors and Messages.” [Online]. Available: https://www.postgresql.org/docs/current/tutorial-transactions.html